

Prerequisite:

AWS CLI should be installed in your system.

1. Install AWS CLI on Ubuntu

Step 1: Update and Install Required Packages

```
sudo apt update
sudo apt install -y unzip curl
```

Step 2: Download and Install AWS CLI

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
```

Step 3: Verify Installation

```
aws --version
```

2. Install AWS CLI on macOS

Method 1: Using Homebrew (Recommended)

```
brew install awscli
```

Method 2: Manual Installation

```
curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
```

```
sudo installer -pkg AWSCLIV2.pkg -target /
```

Verify Installation

```
aws --version
```

3. Install AWS CLI on Windows

Step 1: Download the AWS CLI Installer

- Download from the official AWS website:
<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

Step 2: Install AWS CLI

- Run the downloaded `.msi` installer and follow the setup wizard.

Step 3: Verify Installation

- Open **Command Prompt (cmd)** or **PowerShell** and run:

```
aws --version
```

Configure AWS CLI

After installing AWS CLI on either **Windows** or **Ubuntu**, configure it with your AWS credentials.

`aws configure`

You'll be prompted to enter:

- **AWS Access Key ID:** *Your access key*
- **AWS Secret Access Key:** *Your secret key*
- **Default region:** *(e.g., us-east-1)*
- **Output format:** *(Leave as json or choose text/table)*

Hour 1: Cloud & AWS Essentials

Concepts Covered:

- What is Cloud Computing? (IaaS, PaaS, SaaS)
- AWS Global Infrastructure (Regions, AZs, Edge Locations)
- IAM Basics (Users, Roles, Policies, MFA)
- Cloud Storage (S3, EBS, Object Lifecycle)

1. Cloud Computing (IaaS, PaaS, SaaS)

Cloud computing delivers computing resources (servers, storage, databases, networking, software) over the internet, allowing businesses to scale efficiently.

- **IaaS (Infrastructure as a Service):** Provides virtual machines, storage, and networking (e.g., AWS EC2, Azure Virtual Machines).
- **PaaS (Platform as a Service):** Offers a development environment with pre-configured tools and infrastructure (e.g., AWS Elastic Beanstalk, Google App Engine).
- **SaaS (Software as a Service):** Delivers fully managed software applications via the cloud (e.g., Gmail, Dropbox, Salesforce).

2. AWS Global Infrastructure

AWS operates a global network of data centers to provide scalable cloud services.

- **Regions:** Geographical areas with multiple data centers (e.g., US-East-1, Asia-Pacific).
- **Availability Zones (AZs):** Isolated data centers within a region that provide redundancy and high availability.
- **Edge Locations:** CDN endpoints used for caching content closer to users, improving performance (e.g., CloudFront).

3. IAM Basics (Identity and Access Management)

AWS IAM (Identity and Access Management) controls access to AWS resources securely.

- **Users:** Individual accounts with specific permissions.
- **Roles:** Assigned permissions for AWS services or federated users.
- **Policies:** JSON-based rules defining permissions (e.g., allow EC2 start/stop).
- **MFA (Multi-Factor Authentication):** Adds an extra layer of security using OTP or authentication apps.

4. Cloud Storage (S3, EBS, Object Lifecycle)

AWS provides various storage options for different use cases.

- **S3 (Simple Storage Service)**: Object storage for scalable data storage with different classes (Standard, IA, Glacier).
- **EBS (Elastic Block Store)**: Block storage for EC2 instances, used for high-performance workloads.
- **Object Lifecycle**: Automates transitioning or deleting S3 objects based on rules (e.g., move to Glacier after 30 days).

Hands-on Labs:

1. Setup **AWS Free Tier Account**
2. Create **IAM Users & Roles with MFA**
3. Launch a **Basic EC2 Instance**

step-by-step guide with AWS links to help you navigate through each task:

1. Set Up AWS Free Tier Account

Steps:

1. Go to [AWS Free Tier Sign-Up](#)
2. Click "Create a Free Account"
3. Enter **email**, set **root user credentials**, and verify your **phone number**
4. Enter **billing details** (AWS Free Tier is free, but you need to add a card)
5. Choose **support plan** (select "Basic" to avoid charges)
6. Verify your identity, then **log in to the AWS Console** → [AWS Console](#)

2. Create IAM Users & Roles with MFA

Create an IAM User:

1. Go to [AWS IAM Console](#)
2. Click **Users** → **Add users**
3. Set a **username** and choose **AWS Management Console access**
4. Select **custom password** or generate one
5. Assign **permissions** (Attach existing policies like `AdministratorAccess` or `EC2FullAccess`)
6. Click **Create user**

Enable MFA (Multi-Factor Authentication) for IAM User:

1. Go to [AWS IAM Console](#)
2. Select your user, go to **Security credentials**
3. Click **Manage MFA** → **Enable MFA**
4. Choose **Virtual MFA device** (e.g., Google Authenticator)
5. Scan the QR code and enter two OTPs to activate

Create an IAM Role:

1. Go to [AWS IAM Console](#)
2. Click **Roles** → **Create Role**
3. Select **AWS Service** (e.g., EC2, Lambda)
4. Attach policies (e.g., **AmazonS3ReadOnlyAccess** for S3 access)
5. Name the role and **create it**

3. Launch a Basic EC2 Instance

1. Go to [AWS EC2 Console](#)
2. Click **Launch Instance**
3. Choose **Amazon Linux** (or Ubuntu, Free Tier eligible)
4. Select **t2.micro** instance type (Free Tier)
5. Configure settings (leave default)
6. Create or select an **existing key pair** for SSH access
7. Click **Launch**

.....

Hour 2: EC2 & VPC Networking Basics

Concepts Covered:

- EC2 Basics (Instance Types, Key Pairs, Security Groups)
- VPC Overview (CIDR, Subnets, Route Tables, Internet Gateway)
- Security Groups vs. NACLs

1. EC2 Basics

Amazon EC2 (Elastic Compute Cloud) provides scalable virtual servers (instances) in the AWS cloud.

- **Instance Types:** EC2 instances come in different families optimized for various workloads:
 - **General Purpose** (e.g., t2, t3) – Balanced CPU/memory (Web servers, Dev/Test).
 - **Compute Optimized** (e.g., c5, c6g) – High CPU for processing-intensive apps.
 - **Memory Optimized** (e.g., r5, x2) – High RAM for databases, caching.
 - **Storage Optimized** (e.g., i3, d2) – High-speed disk access.
 - **GPU Optimized** (e.g., p3, g5) – AI/ML and graphics processing.
- **Key Pairs:** SSH key pairs (.pem files) authenticate secure access to EC2 instances. You must create/download a key pair when launching an EC2 instance.
- **Security Groups:** Act as virtual firewalls that control inbound/outbound traffic for instances. Rules are stateful (i.e., if you allow inbound traffic, the response is automatically allowed).

2. VPC (Virtual Private Cloud) Overview

A **VPC (Virtual Private Cloud)** is a logically isolated network in AWS where you deploy resources.

- **CIDR (Classless Inter-Domain Routing):** Defines the IP range for your VPC (e.g., 10.0.0.0/16 allows 65,536 IPs).
- **Subnets:** Divide a VPC into smaller networks. Can be **public** (internet-accessible) or **private** (internal use).
- **Route Tables:** Define how traffic is routed within a VPC (e.g., allowing private subnets to access the internet via NAT Gateway).
- **Internet Gateway:** A component that allows internet access for instances in public subnets.

3. Security Groups vs. NACLs (Network Access Control Lists)

Both control traffic flow in a VPC but serve different purposes:

Feature	Security Groups	NACLs
Level	Instance-Level	Subnet-Level
Stateful?	Yes (responses to allowed traffic are automatically allowed)	No (each request needs an explicit rule)
Default Behavior	Deny all inbound, allow all outbound	Allow all inbound/outbound
Rules	Only allow rules	Allow & deny rules
Evaluation	Checks all rules before allowing	Evaluates rules in numbered order

📌 Use **Security Groups** for instance-level security (e.g., allow SSH on port 22).

📌 Use **NACLs** for subnet-wide control (e.g., block an IP range for an entire subnet).

Hands-on Labs:

1. Connect to an **EC2 instance using SSH**
2. Configure **VPC with Public & Private Subnets**
3. Modify **Security Groups & Test Access Restrictions**

1. Connect to an EC2 Instance using SSH

Steps to Connect to EC2 via SSH

1. Find Your EC2 Public IP

- Go to [AWS EC2 Console](#)
- Select your running instance and copy the **Public IPv4 Address**

Connect via SSH (Linux/macOS)

Open a terminal and run:

```
ssh -i your-key.pem ec2-user@your-ec2-public-ip
```

(Replace `your-key.pem` with your downloaded key and `your-ec2-public-ip` with the copied IP)

Connect via SSH (Windows - PuTTY)

- Convert `.pem` to `.ppk` using PuTTYgen
- Open **PuTTY** → Enter **Public IP** under "Host Name"
- Navigate to **Connection > SSH > Auth** → Browse & select `.ppk` file
- Click **Open**

2. Configure VPC with Public & Private Subnets

Steps to Create a VPC with Public & Private Subnets

1. Go to [VPC Console](#)
2. Click "**Create VPC**" → Choose "**VPC with Subnets**"
3. Enter **VPC Name** (e.g., MyVPC) and **CIDR Block** (e.g., `10.0.0.0/16`)
4. Add **Two Subnets**:
 - **Public Subnet** (e.g., `10.0.1.0/24`, enable Auto-Assign Public IP)
 - **Private Subnet** (e.g., `10.0.2.0/24`)
5. **Attach an Internet Gateway**
 - Go to **Internet Gateways** → **Create** → Attach to **MyVPC**
 - Modify the **Route Table** for the public subnet to route `0.0.0.0/0` via Internet Gateway
6. **Add a NAT Gateway for Private Subnet** (Optional)
 - Go to **NAT Gateways** → **Create** → Associate with Public Subnet

3. Modify Security Groups & Test Access Restrictions

Modify Security Groups for EC2 Access

1. Go to [EC2 Security Groups](#)
2. Select the **security group** assigned to your EC2 instance
3. Modify **Inbound Rules**:
 - **Allow SSH (port 22)** only for your IP (My IP option)
 - **Allow HTTP (port 80)** for all (`0.0.0.0/0`)

Test Access Restrictions

1. **From your local system:**

- Try SSH access using `ssh -i your-key.pem ec2-user@your-ec2-public-ip`
- Open a browser and enter `http://your-ec2-public-ip` (should work if HTTP is allowed)

2. **Block SSH & Test Denial:**

- Remove the **SSH rule** in Security Group
- Try to SSH again (it should fail)

.....

Hour 3: Linux Essentials for SREs

Concepts Covered:

- Linux File System & Directory Structure
- Essential Linux Commands (ls, cat, grep, find, chmod)
- Process Management (ps, top, kill, journalctl)
- Log Files & Troubleshooting Basics

1. Linux File System & Directory Structure

The Linux file system is organized in a hierarchical tree structure, with the root directory (/) at the top. It follows the FHS (Filesystem Hierarchy Standard), ensuring consistency across distributions. Each directory has a specific role, making Linux efficient and modular.

Linux follows a hierarchical directory structure with / (root) at the top.

Director y	Purpose
/bin	Essential system binaries (e.g., <code>ls</code> , <code>cat</code>)
/boot	Bootloader files (e.g., <code>grub</code>)
/dev	Device files (e.g., disks, USB)
/etc	Configuration files (e.g., <code>/etc/passwd</code>)
/home	User home directories (<code>/home/username/</code>)

<code>/var</code>	Logs & variable data (<code>/var/log/</code>)
<code>/tmp</code>	Temporary files (deleted on reboot)
<code>/usr</code>	User-installed programs & libraries
<code>/proc</code>	Virtual filesystem for system info (e.g., CPU, memory)

2. Essential Linux Commands

Linux provides a powerful command-line interface (CLI) for file management, searching, and permissions. Here are some commonly used commands:

- `ls`, `cat`, `touch`, `top`, `grep`, `chmod`, `mkdir`, `rm` etc.

3. Process Management

Linux is a multi-tasking OS, and managing processes effectively is crucial. A process is a running instance of a program.

Viewing Processes : `ps aux`, `top`, `top`

Managing Processes: `kill <PID>`, `kill -9 <PID>`

System Logs (for Troubleshooting): `journalctl -xe` (view system log), `systemctl status <service>` (check service status)

4. Log Files & Troubleshooting Basics

Logs are essential for debugging issues and monitoring system activity. Linux maintains various logs under `/var/log/`.

Log File	Purpose
<code>/var/log/syslog</code>	General system logs
<code>/var/log/auth.log</code>	Authentication logs (login attempts, sudo usage)
<code>/var/log/kern.log</code>	Kernel messages
<code>/var/log/apache2/access.log</code>	Web server access logs
<code>/var/log/mysql/error.log</code>	Database error logs

Monitoring Logs

```
tail -f /var/log/syslog # View real-time log updates
```

```
less /var/log/auth.log # Read authentication logs
```

Checking Disk & Memory Usage

```
df -h # Check disk space
```

```
free -m # Check memory usage
```

Hands-on Labs:

1. Execute basic **Linux commands on an EC2 instance**
2. Modify **File Permissions & Manage Processes**
3. Install and run **Nginx or Apache on EC2**

Step 1: Connect to an EC2 Instance via SSH

1. Go to AWS Console → [AWS EC2 Dashboard](#)
2. Select your instance and click **Connect**.
Use SSH to connect from your terminal:

```
ssh -i your-key.pem ec2-user@your-ec2-public-ip
```

Step 2: Execute Basic Linux Commands

Once connected, try running these commands:

Check system information, File management, Process management

```
uname -a # Show OS and kernel details
```

```
df -h # Show disk usage
```

```
free -m # Show memory usage
```

```
ls -l # List files with details
```

```
mkdir test # Create a new directory
```

```
touch file.txt # Create an empty file
```

```
rm file.txt # Remove a file
```

```
ps aux      # Show running processes
top         # Real-time process monitor
kill -9 <PID> # Kill a specific process
```

Step 3: Modify File Permissions

Change ownership:

```
sudo chown ec2-user:ec2-user file.txt
```

Modify permissions:

```
chmod 755 script.sh # Give execute permission
```

Step 4: Install and Run Nginx or Apache

For Nginx

1. Install Nginx:

```
sudo amazon-linux-extras enable nginx1
sudo yum install nginx -y
```

Start and Enable Nginx:

```
sudo systemctl start nginx
sudo systemctl enable nginx
```

Check status:

```
systemctl status nginx
```

1. Allow HTTP Traffic in Security Groups:

- Go to **EC2 Dashboard** → **Security Groups**.
- Select your security group.
- Add a rule for **HTTP (port 80)** and **HTTPS (port 443)**.

2. Access Nginx via browser:

Open `http://your-ec2-public-ip`

Step 5: Modify Security Groups & Test Access

1. Go to AWS EC2 Console → [Security Groups](#)
2. Select your instance's security group.
3. Add rules for:

- **SSH (port 22)** → Your IP
 - **HTTP (port 80)** → Anywhere
 - **HTTPS (port 443)** → Anywhere
4. **Test:**
- Open a browser and visit `http://your-ec2-public-ip`
 - Try stopping the web server:


```
sudo systemctl stop nginx
```
 - Refresh the browser to see if access is restricted.

Hour 4: AWS Monitoring & Troubleshooting

Concepts Covered:

- AWS CloudWatch Basics (Metrics, Logs, Alerts)
- Troubleshooting SSH Access & Security Groups
- Automating EC2 & S3 Operations with AWS CLI

1. AWS CloudWatch Basics (Metrics, Logs, Alerts)

Amazon **CloudWatch** is a monitoring service for AWS resources and applications. It helps track performance, detect issues, and trigger automated responses.

Metrics:

- Collects and stores real-time performance data from AWS services.
- Examples: EC2 CPU usage, memory utilization, S3 bucket size.

Viewable in the **CloudWatch Dashboard**.

```
aws cloudwatch list-metrics --namespace AWS/EC2
```

Logs:

- Stores and manages logs from AWS services (e.g., EC2, Lambda).
- Helps debug errors and analyze application performance.
- Example: EC2 system logs are stored in **CloudWatch Logs**.

Alerts (Alarms):

- Sends notifications when metrics exceed defined thresholds.
- Example: Trigger an alert if EC2 CPU usage exceeds 80%.
- Can be integrated with **SNS (Simple Notification Service)** for email/SMS alerts.

2. Troubleshooting SSH Access & Security Groups

If you can't connect to your EC2 instance via SSH, follow these steps:

1. Verify Security Group Rules

- Go to AWS Console → [EC2 Security Groups](#).
- Check **Inbound Rules**:
 - **Port 22 (SSH)** should be open to **your IP** (0 . 0 . 0 . 0 / 0 for public access, not recommended).

2. Check EC2 Key Pair & SSH Command

- Ensure you are using the correct **.pem key**:

```
ssh -i your-key.pem ec2-user@your-ec2-public-ip
```

- Fix permissions on the key file:

```
chmod 400 your-key.pem
```

3. Verify EC2 Instance State

- Go to [EC2 Dashboard](#) and check if your instance is **running**.
- If the instance is stuck, restart it:

```
aws ec2 reboot-instances --instance-ids i-xxxxxxx
```

4. Check System Logs for Errors

- Use EC2 Serial Console:

```
aws ec2 get-console-output --instance-id i-xxxxxxx
```

3. Automating EC2 & S3 Operations with AWS CLI

AWS CLI allows you to automate EC2 and S3 tasks from the command line.

EC2 Operations

- **Start an EC2 instance:**

```
aws ec2 start-instances --instance-ids i-xxxxxxx
```

- **Stop an EC2 instance:**

```
aws ec2 stop-instances --instance-ids i-xxxxxxx
```

S3 Operations

- **List all S3 buckets:**

```
aws s3 ls
```

- **Upload a file to S3:**

```
aws s3 cp file.txt s3://your-bucket-name/
```

- **Download a file from S3:**

```
aws s3 cp s3://your-bucket-name/file.txt .
```

- **Delete an S3 object:**

```
aws s3 rm s3://your-bucket-name/file.txt
```

Hands-on Labs:

1. Setup **CloudWatch Monitoring for EC2 Metrics**

2. Troubleshoot **Common EC2 Connection Issues**
3. Use **AWS CLI to Start/Stop EC2 & S3 Operations**

1. Setup CloudWatch Monitoring for EC2 Metrics

AWS CloudWatch monitors EC2 instances by collecting system metrics such as CPU usage, disk I/O, and network activity.

Steps to Enable CloudWatch Monitoring for EC2:

1. Go to AWS Console → [EC2 Dashboard](#)
2. Select your instance → Click on **Monitoring** tab.
3. Click **Manage detailed monitoring** → Enable **Detailed Monitoring** (for 1-minute interval data).
4. Go to AWS Console → [CloudWatch Dashboard](#)
5. Navigate to **Metrics** → Select **EC2**
6. Choose a metric (e.g., **CPUUtilization**) and create a graph.
7. To set an alarm:
 - Go to **Alarms** → **Create Alarm**
 - Select **EC2 metric (CPU Utilization, Memory, Disk, etc.)**
 - Set threshold (e.g., CPU > 80%)
 - Configure **SNS notification** for alerts.

2. Troubleshoot Common EC2 Connection Issues

If you're unable to SSH into an EC2 instance, follow these steps.

1. Check Security Groups & SSH Port

1. Go to AWS Console → [EC2 Security Groups](#).
2. Select your **instance's security group**.
3. Verify **Inbound Rules**:
 - **SSH (Port 22)** → Set source to **your IP (not 0.0.0.0/0 for security reasons)**.

2. Verify Key Pair & SSH Command

Ensure you have the correct **.pem key** and permissions:

```
chmod 400 your-key.pem
ssh -i your-key.pem ec2-user@your-ec2-public-ip
```

3. Check Instance Status & Public IP

- Check instance status in EC2 Console → Instance State: Running
- Find the Public IPv4 address under Instance Summary.
- Confirm EC2 instance is reachable:

```
ping your-ec2-public-ip
```

4. View EC2 System Logs

- Go to AWS Console → [EC2 Instances](#)
- Select the instance → Click Actions → Monitor and Troubleshoot → Get system log
- To check logs via CLI:

```
aws ec2 get-console-output --instance-id i-xxxxxxx
```

3. Use AWS CLI to Start/Stop EC2 & S3 Operations

Start & Stop EC2 Instances

- Start an EC2 instance: bash

```
aws ec2 start-instances --instance-ids i-xxxxxxx
```

- Stop an EC2 instance:

```
aws ec2 stop-instances --instance-ids i-xxxxxxx
```

S3 Bucket Operations

Steps to Manage S3 in the Console:

1. Go to the AWS S3 Console → [S3 Dashboard](#)
2. Click **Buckets** → Select the bucket you want to manage.
3. Perform operations:
 - **Upload a file:** Click **Upload** → Select file → Click **Upload**.
 - **Download a file:** Click on the file → Click **Download**.
 - **Delete a file:** Select the file → Click **Delete** → Confirm deletion.

- **List all S3 buckets:**

```
aws s3 ls
```

- **Upload a file to S3:**

```
aws s3 cp file.txt s3://your-bucket-name/
```

- **Download a file from S3:**

```
aws s3 cp s3://your-bucket-name/file.txt .
```

- **Delete an S3 object:**

```
aws s3 rm s3://your-bucket-name/file.txt
```

Task	AWS Console Navigation	AWS CLI Command
Start EC2 instance	EC2 Dashboard → Select instance → Instance State → Start	<code>aws ec2 start-instances --instance-ids i-xxxxxxx</code>
Stop EC2 instance	EC2 Dashboard → Select instance → Instance State → Stop	<code>aws ec2 stop-instances --instance-ids i-xxxxxxx</code>
List S3 Buckets	S3 Dashboard → View Buckets	<code>aws s3 ls</code>
Upload File to S3	S3 Dashboard → Select Bucket → Upload	<code>aws s3 cp file.txt s3://your-bucket-name/</code>
Download File from S3	S3 Dashboard → Select File → Download	<code>aws s3 cp s3://your-bucket-name/file.txt .</code>

Delete File from S3	S3 Dashboard → Select File → Delete	<code>aws s3 rm s3://your-bucket- name/file.txt</code>
------------------------	--	--